

Title Page

CODSOFT

Artificial Intelligence Internship Report

Project Title

Rule-Based AI Chatbot

Submitted in partial fulfillment of the requirements for the

CodSoft Artificial Intelligence Internship

Submitted By

Name: Pavani S K Reddy

GitHub: <https://github.com/pavanireddy06>

Project: Task 1 – Rule-Based AI Chatbot

Technology: Python

Internship Organization: CodSoft

Submission Date: August 2026

Certificate

This is to certify that **Pavani S K Reddy** has successfully completed the project titled "**Rule-Based AI Chatbot**" as part of the **CodSoft Artificial Intelligence Internship**.

The project demonstrates the implementation of a chatbot using predefined rules and pattern matching techniques in Python. During the project, concepts such as modular programming, file handling, input validation, and basic Natural Language Processing (NLP) were applied to build an interactive chatbot capable of responding to user queries.

This report documents the development process, implementation details, and outcomes of the project.

Acknowledgement

I would like to express my sincere gratitude to **CodSoft** for providing me with the opportunity to participate in the Artificial Intelligence Internship.

This internship allowed me to gain practical knowledge in Python programming and Artificial Intelligence through hands-on project development.

I also thank my mentors, faculty members, and everyone who supported me throughout the completion of this project. Their guidance and encouragement helped me improve my technical and problem-solving skills.

Finally, I would like to thank my family and friends for their continuous motivation and support during this internship.

Abstract

The **Rule-Based AI Chatbot** is a Python-based conversational application designed to interact with users using predefined rules and keyword-based pattern matching.

Unlike machine learning chatbots, this chatbot does not require training data. Instead, it processes user input by matching keywords and returning appropriate predefined responses.

The chatbot includes several practical features such as:

- Greeting users
- Providing date and time
- Knowledge base for Python, Java, AI, SQL, Git, HTML, CSS, JavaScript, and React
- Safe calculator
- Random programming jokes
- Motivational quotes
- Technology facts
- Chat history logging
- Conversation statistics
- Typo detection
- Help menu

The project demonstrates fundamental Artificial Intelligence concepts including rule-based reasoning, modular software design, file handling, and pattern matching while maintaining simplicity and ease of understanding.

Table of Contents

1. Introduction

2. Objectives

3. Problem Statement

4. Tools & Technologies

5. System Architecture

6. Modules

7. Implementation

8. Features

9. Output Screenshots

10. Challenges Faced

11. Learning Outcomes

12. Future Enhancements

13. Conclusion

References

1. Introduction

Artificial Intelligence has become one of the fastest-growing fields in computer science. One of its simplest applications is the development of chatbots capable of interacting with users.

A rule-based chatbot uses predefined rules rather than machine learning algorithms. It analyzes user input, identifies keywords, and generates appropriate responses based on programmed conditions.

The objective of this project is to understand the fundamentals of chatbot development by implementing a conversational system using Python.

The chatbot demonstrates how Artificial Intelligence can automate basic communication tasks using logical decision-making without requiring large datasets or model training.

2. Objectives

The main objectives of this project are:

- Develop a chatbot using Python.
- Implement rule-based conversation using keyword matching.
- Understand basic Natural Language Processing concepts.
- Practice modular programming.
- Learn file handling techniques.
- Store conversation history.
- Improve user interaction through a friendly interface.
- Develop a project suitable for GitHub and internship submission.

3. Problem Statement

Many beginners find Artificial Intelligence difficult because modern AI systems rely heavily on machine learning and large language models.

The purpose of this project is to build a simple chatbot that demonstrates the fundamental concept of AI decision-making using predefined rules.

The chatbot should:

- Accept user input.
- Identify keywords.
- Respond appropriately.

- Handle unknown commands.
- Perform simple calculations.
- Store conversation history.
- Provide useful information interactively.

4. Tools & Technologies

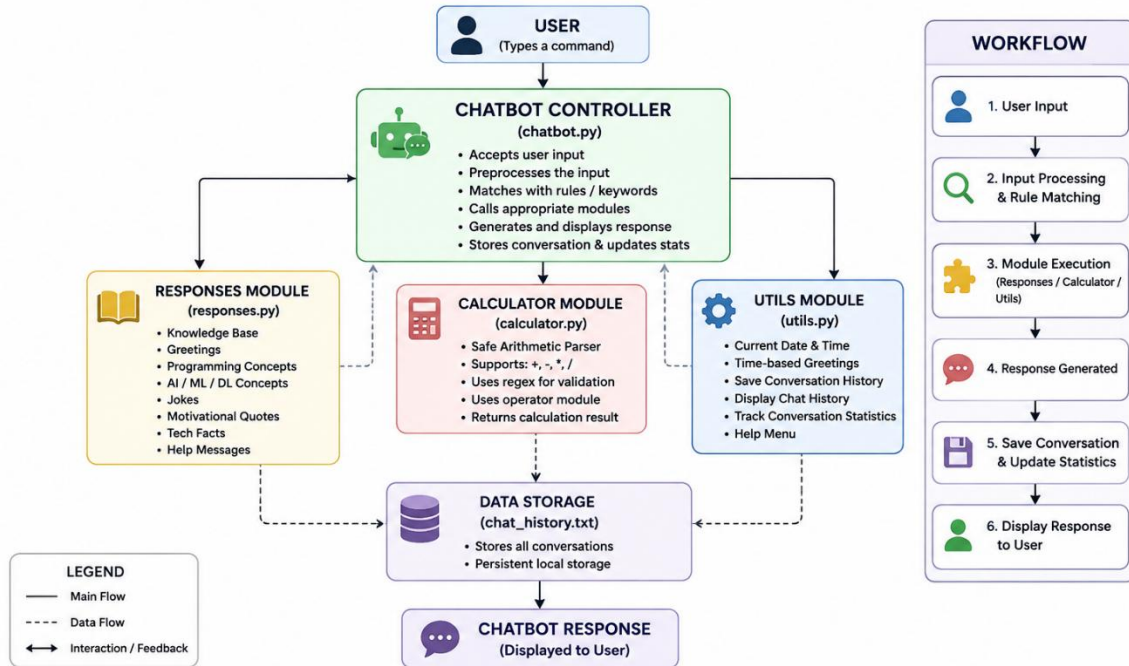
Tool	Purpose
Python	Programming Language
VS Code	Code Editor
Git	Version Control
GitHub	Source Code Hosting
Colorama	Colored Terminal Output
Datetime Module	Date & Time
DiffLib	Typo Detection
Operator Module	Calculator Operations
Regular Expressions (re)	Expression Validation

5. System Architecture

The Rule-Based AI Chatbot follows a modular architecture in which each module is responsible for a specific task. The chatbot receives user input, processes it using predefined rules, and generates an appropriate response. If the user requests a calculation, the request is passed to the calculator module. Similarly, utility functions handle operations such as displaying the current date and time, storing chat history, and tracking conversation statistics.

SYSTEM ARCHITECTURE

RULE-BASED AI CHATBOT



Workflow

1. User enters a command.
2. The chatbot converts the input to lowercase.
3. The program checks predefined rules.
4. If the command is recognized:
 - Execute the corresponding module.
5. If the command is not recognized:
 - Suggest a similar command using typo detection.
6. Display the response.
7. Save the conversation in chat_history.txt.
8. Wait for the next user input.

6. Modules

The chatbot is divided into multiple modules to improve readability, maintainability, and scalability.

Module 1 – chatbot.py

This is the main controller of the application.

Responsibilities

- Accept user input
- Identify commands
- Display responses
- Call other modules
- Handle conversation flow
- Save chat history
- Maintain conversation statistics

Module 2 – responses.py

This module stores all predefined chatbot responses.

Responsibilities

- Greeting messages
- Programming jokes
- Motivational quotes
- Technology facts
- AI knowledge
- Python knowledge
- Java knowledge
- SQL knowledge
- Git & GitHub information
- HTML, CSS, JavaScript, React information

Using a separate file makes it easier to add new commands without modifying the main program.

Module 3 – calculator.py

This module performs arithmetic calculations.

Supported Operations

- Addition
- Subtraction
- Multiplication
- Division

The calculator uses Python's operator module and regular expressions for validation instead of eval(), making it safer and more reliable.

Module 4 – utils.py

This module contains utility functions.

Functions

- Current date
- Current time
- Time-based greeting
- Help menu
- Save conversation history
- Display history
- Track chatbot statistics

7. Implementation

The chatbot is implemented using Python and follows a modular programming approach.

The application starts by displaying a welcome screen and requesting the user's name. Based on the current time, it greets the user with an appropriate message such as "Good Morning" or "Good Evening."

After initialization, the chatbot continuously waits for user input using an infinite loop.

The entered text is converted to lowercase to avoid case sensitivity.

The chatbot then checks whether the command matches any predefined keywords.

If a matching keyword is found, the corresponding response is generated.

For example:

- hello → Greeting

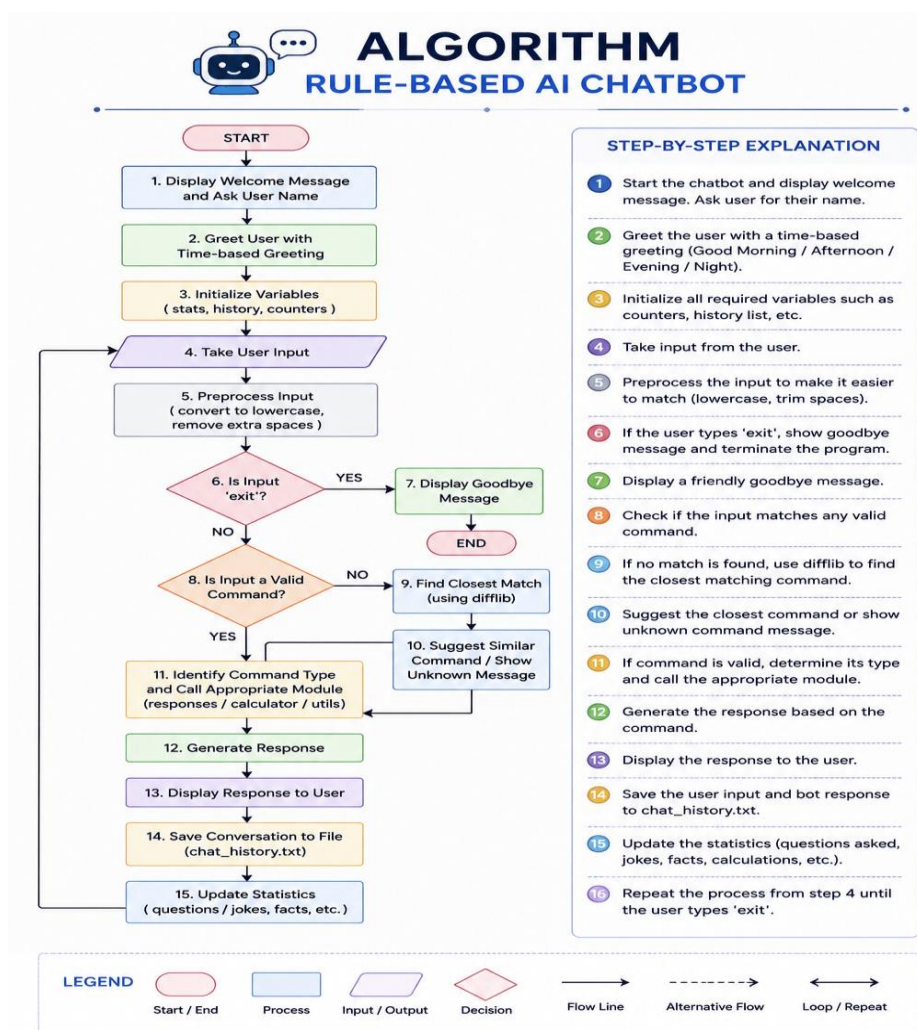
- python → Python information
- joke → Random programming joke
- calculate 20+30 → Calculator module
- history → Display previous conversations
- stats → Show conversation statistics

If the user enters an incorrect command such as **pyhton**, the chatbot uses Python's `difflib.get_close_matches()` function to suggest the closest valid command.

Every interaction is automatically stored in `chat_history.txt`, allowing previous conversations to be reviewed.

The chatbot continues responding until the user enters the **exit** command.

Algorithm



8. Features

The chatbot includes the following features:

User Interaction

- User name recognition
- Time-based greetings
- Friendly responses
- Help menu

Knowledge Base

- Python
- Java
- Artificial Intelligence
- Machine Learning
- Deep Learning
- SQL
- Git
- GitHub
- HTML
- CSS
- JavaScript
- React

Utility Features

- Current date
- Current time
- Safe calculator
- Conversation statistics
- Chat history

Entertainment Features

- Programming jokes
- Motivational quotes

- Technology facts

Smart Features

- Typo correction
- Unknown command handling
- Modular programming
- Colorful terminal interface

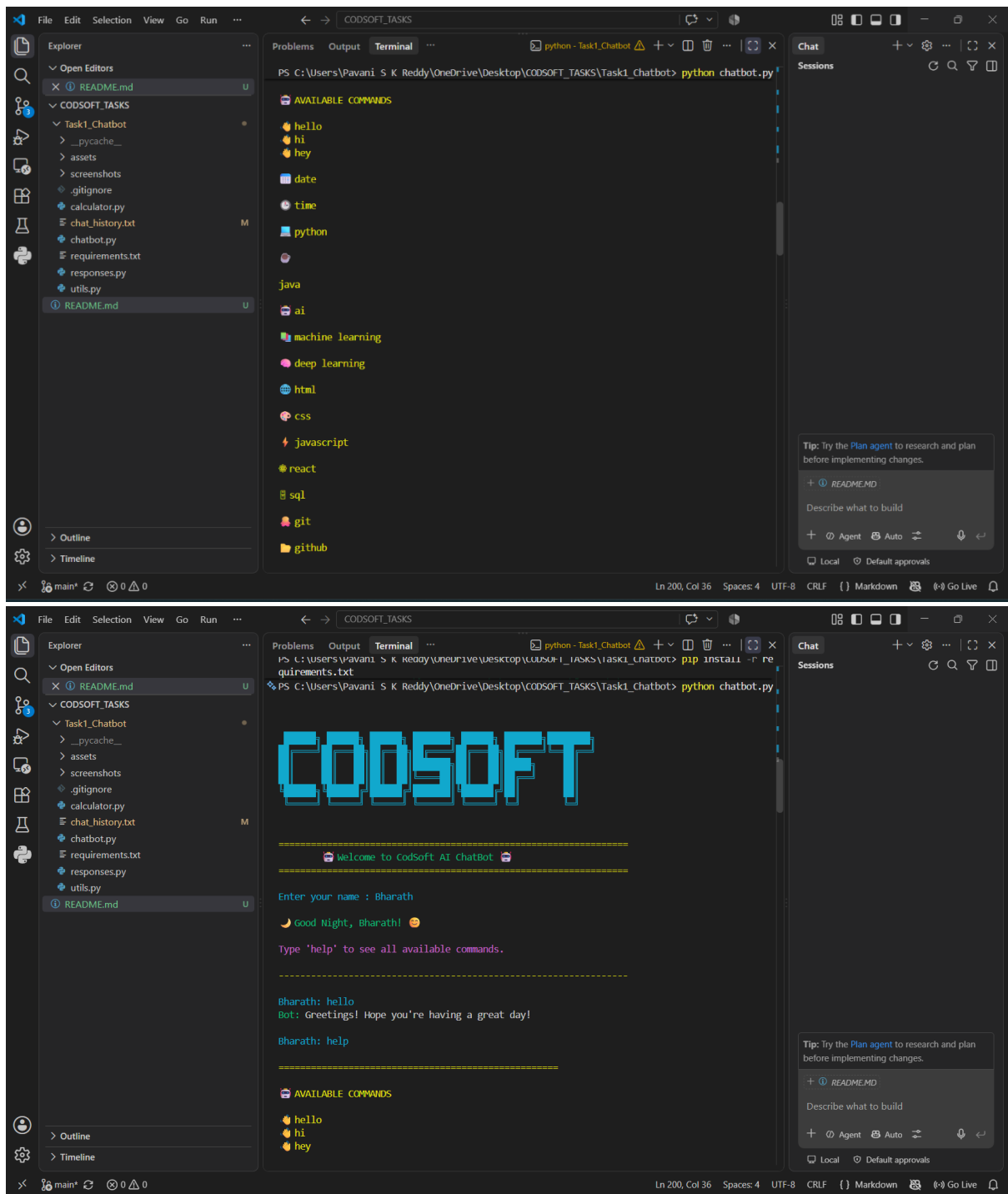
Advantages

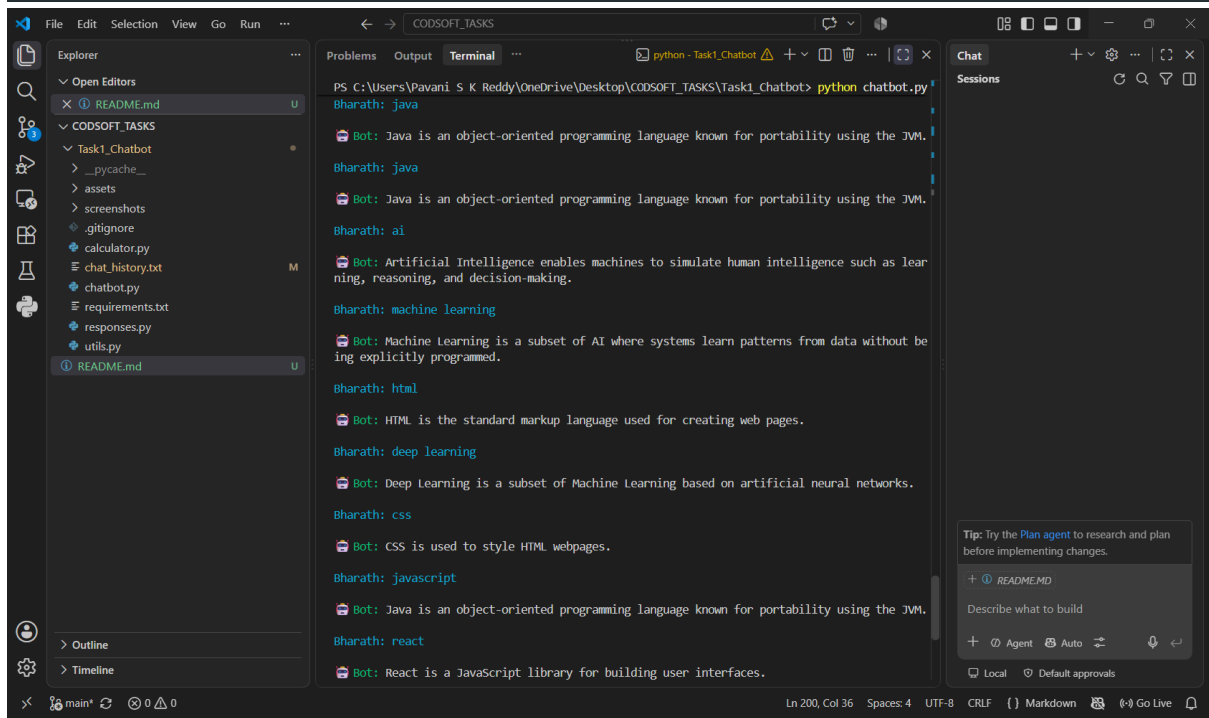
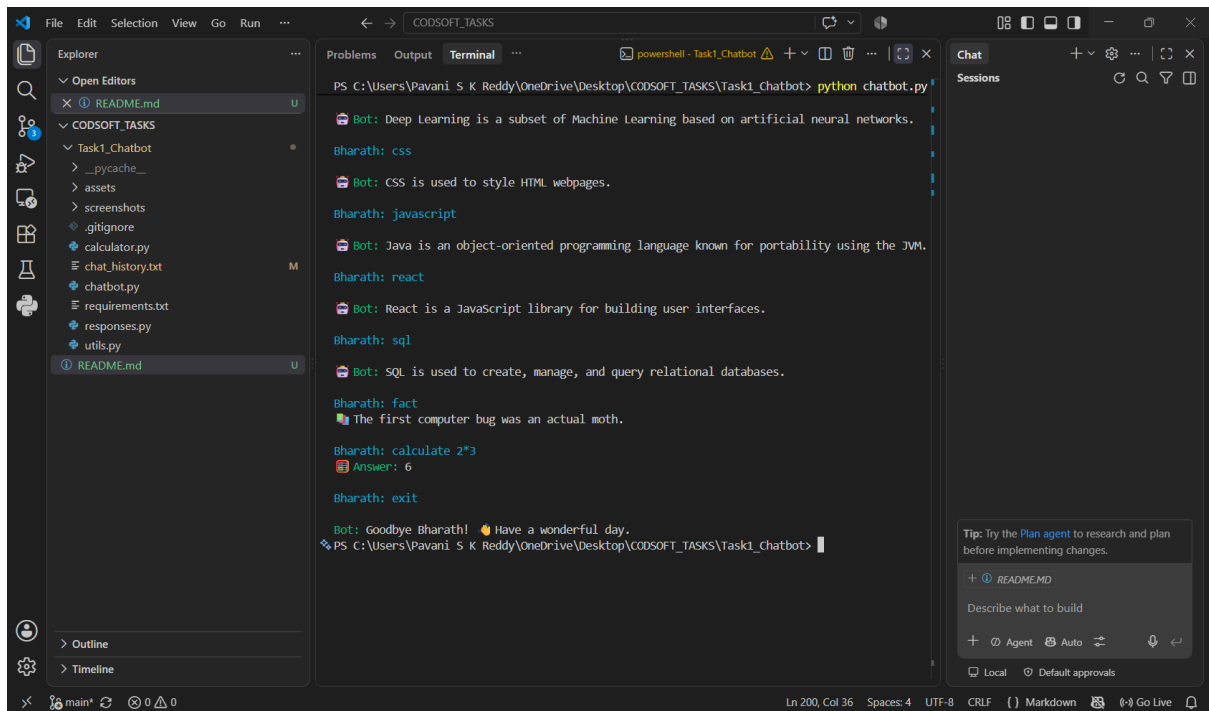
- Easy to understand
- Beginner-friendly
- Modular design
- Fast execution
- Low memory usage
- No internet connection required
- Easy to extend with additional commands
- Safe calculator implementation

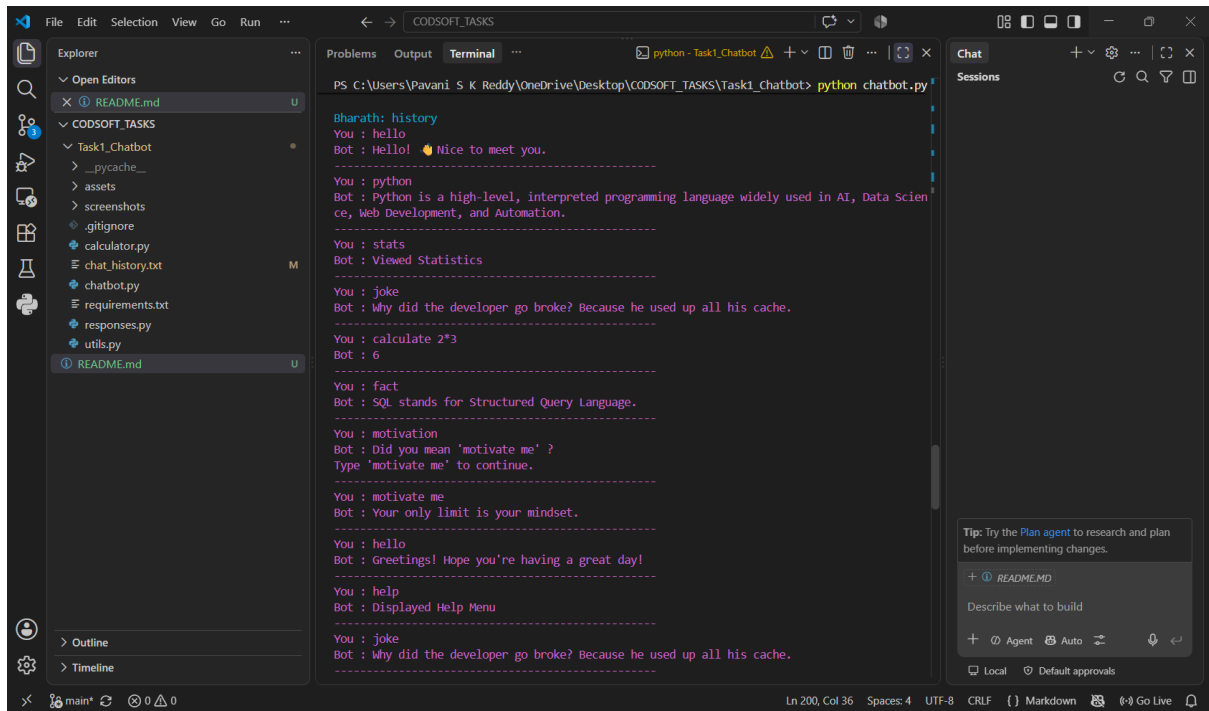
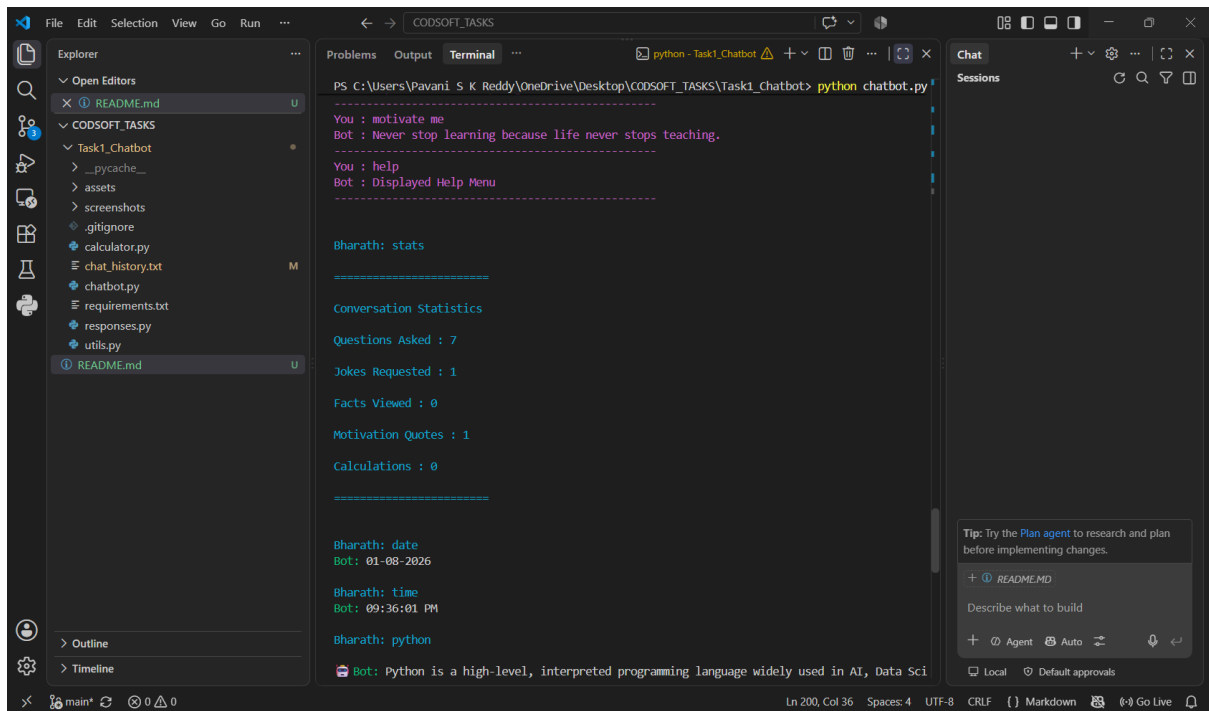
Limitations

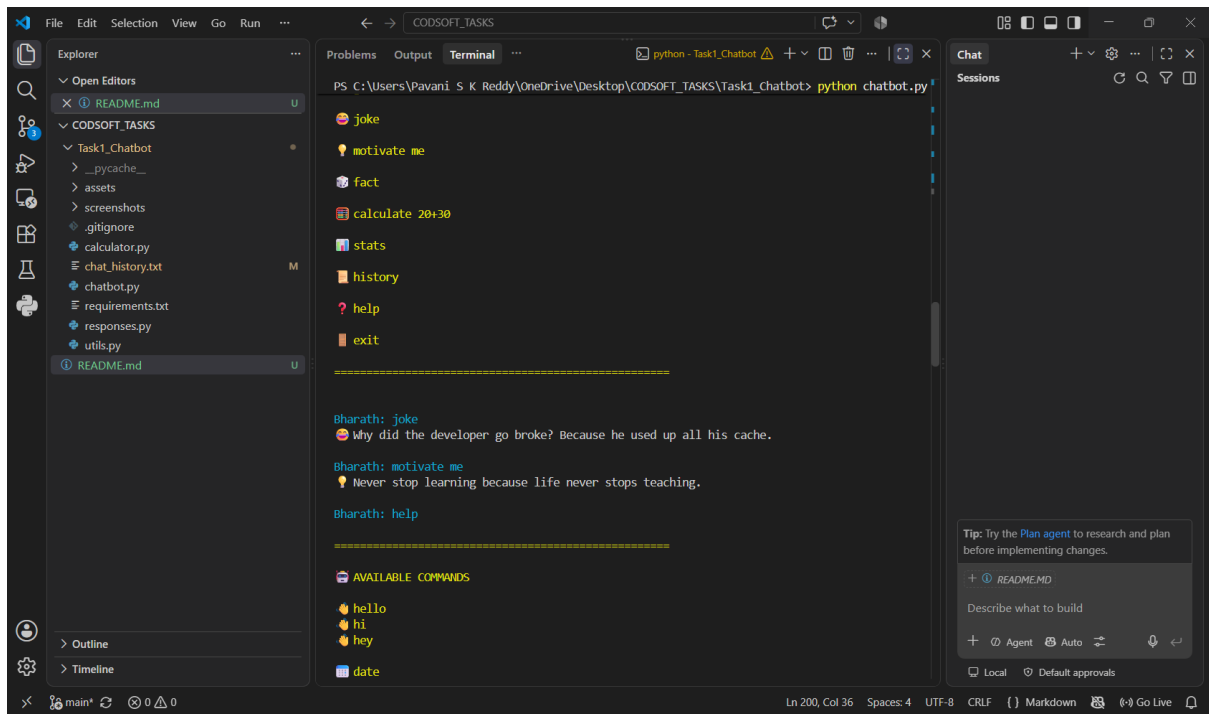
- Uses predefined rules only
- Cannot learn from conversations
- Limited understanding of natural language
- Does not use machine learning
- Cannot answer questions outside the predefined knowledge base

9. Output Screenshots









10. Challenges Faced

During the development of the Rule-Based AI Chatbot, several challenges were encountered.

Challenge 1 – Designing Rule-Based Responses

Creating predefined responses that covered different types of user queries required careful planning. Multiple keywords had to be considered so that the chatbot could recognize user intent accurately.

Challenge 2 – Safe Calculator Implementation

Initially, using Python's `eval()` function appeared to be the easiest solution for implementing the calculator. However, `eval()` can execute arbitrary Python code and poses security risks.

To address this, a safer calculator was implemented using regular expressions and Python's operator module.

Challenge 3 – Organizing the Project

As the project grew, maintaining all functionality in a single file became difficult.

This challenge was resolved by dividing the application into separate modules such as:

- `chatbot.py`
- `responses.py`

- calculator.py
- utils.py

This improved readability and maintainability.

Challenge 4 – Managing Chat History

Another challenge was storing user conversations efficiently.

This was solved by implementing file handling techniques using Python's built-in file operations.

Challenge 5 – Handling Incorrect Commands

Users often enter commands with spelling mistakes.

To improve user experience, typo detection was implemented using Python's difflib module.

11. Learning Outcomes

Through this project, the following technical skills were learned and strengthened:

Programming Skills

- Python Programming
- Functions
- Conditional Statements
- Loops
- Dictionaries
- Lists
- String Manipulation

Software Development Skills

- Modular Programming
- Code Reusability
- Project Organization
- File Handling

- Error Handling

Artificial Intelligence Concepts

- Rule-Based Systems
- Pattern Matching
- Decision Making
- Keyword Recognition
- Basic Natural Language Processing

Development Tools

- Visual Studio Code
- Git
- GitHub
- Command Line Interface

Professional Skills

- Documentation
- Debugging
- Problem Solving
- Version Control
- Software Design

12. Future Enhancements

Although the chatbot successfully fulfills the internship requirements, several improvements can be made in future versions.

Proposed Enhancements

- Voice Recognition
- Speech-to-Text
- Text-to-Speech

- Graphical User Interface (GUI)
- Database Integration
- User Authentication
- Weather Information API
- News API Integration
- Wikipedia Search
- Multi-language Support
- Machine Learning-based Chatbot
- OpenAI API Integration
- Sentiment Analysis
- Voice Assistant Features

These enhancements would improve usability, scalability, and intelligence while expanding the chatbot beyond a rule-based system.

13. Conclusion

The **Rule-Based AI Chatbot** was successfully designed and implemented using Python as part of the **CodSoft Artificial Intelligence Internship**.

The chatbot demonstrates how predefined rules and keyword-based pattern matching can be used to simulate basic conversational behavior without relying on machine learning models.

Throughout the project, various Python concepts such as modular programming, file handling, regular expressions, conditional statements, and user interaction were applied to create a structured and user-friendly application.

The project not only fulfills the internship task requirements but also serves as a strong foundation for understanding chatbot development and the fundamentals of Artificial Intelligence.

This internship project has significantly enhanced practical programming skills, software development practices, and confidence in building real-world AI applications.

References

1. Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>
2. Git Documentation. <https://git-scm.com/doc>
3. GitHub Documentation. <https://docs.github.com/>
4. Colorama Documentation. <https://pypi.org/project/colorama/>
5. CodSoft Artificial Intelligence Internship Task Guide.